Mobile Integration Guide: API Encryption & Authentication

This document describes two independent mechanisms the mobile client must implement to talk to the .NET API:

1. **Payload encryption** (AES-256-CBC) for endpoints that opt in
2. **Bearer token authentication** using a token returned at login

These mechanisms are orthogonal — a call may require one, both, or neither, depending on the endpoint. This is a reference document; it describes the wire contract, not a specific mobile framework's implementation.

Section 0: Rollout Status — Read This First

Nothing is enforced yet. The server currently accepts requests regardless of whether they're encrypted and regardless of whether they carry a valid bearer token — nothing is rejected today for missing either one. The server is only logging non-compliant calls internally, to track client adoption ahead of enforcement.

Practical implications:

- The mobile team can integrate and test this at their own pace right now without breaking any existing functionality — old, non-compliant calls still work during this period.
- That said, **treat this integration as required homework, not optional polish.** Enforcement will be turned on at a later date, at which point unencrypted calls (on endpoints that require it) and unauthenticated calls will start failing outright.
- **The exact cutover date is not yet set — confirm with the backend team** before assuming a timeline or deprioritizing this work.

Section 1: Payload Encryption

1.1 Cipher Specification

Parameter	Value
Algorithm	AES-256-CBC
Padding	PKCS7
Key	32-byte (256-bit) UTF-8 string, used **directly as raw key bytes**
Key derivation	None — no KDF, no hashing, no salting. The key string's bytes *are* the AES key.
IV	16 random bytes, generated fresh for **every** encryption call

The 32-byte key value ("the shared AES key") will be supplied out of band by the backend team. It must be stored securely on-device and never hardcoded in source control in production builds.

IV reuse is not acceptable. A new random IV must be generated per encryption operation, including when encrypting multiple requests with identical bodies.

1.2 Wire Format

The IV and ciphertext are each base64-encoded independently, then joined with a single colon:

```
<ivBase64>:<cipherTextBase64>
```

This string is then wrapped in a JSON envelope for both requests and responses:

```json
{ "payload": "<ivBase64>:<cipherTextBase64>" }
```

The plaintext that gets encrypted is the original JSON request/response body, UTF-8 encoded before encryption. In other words, the flow is:

```
originalBodyJson (UTF-8 bytes)
  → AES-256-CBC encrypt (random IV, shared key)
  → base64(iv) + ":" + base64(ciphertext)
  → { "payload": "<that string>" }
```

Decryption on the client (for encrypted responses) reverses this: split on the colon, base64-decode both parts, decrypt with the shared key and the given IV, UTF-8-decode the result, parse as JSON.

1.3 When Encryption Applies

Encryption is **opt-in per request**, controlled by a header:

```
X-Encrypted: true
```

- If the client sends this header, both the request body **and** the expected response body are the encrypted envelope described above.
- If the client does **not** send this header, the request and response bodies are plain, unencrypted JSON.
- This allows certain endpoints (health checks, external/third-party integrations, etc.) to opt out of encryption entirely.

Client responsibility: the mobile app must explicitly set `X-Encrypted: true` on every call it wants encrypted, and must be prepared to send and parse plain JSON on any call where it omits the header. There is no global mode — this is decided per request.

1.4 Multipart / File Upload Exemption

Multipart/form-data requests (file uploads, etc.) must **never** be encrypted:

- Do **not** encrypt multipart bodies.
- Do **not** set `X-Encrypted: true` on multipart requests.
- These requests pass through as standard `multipart/form-data`.

1.5 Decryption Failure Behavior

If the server cannot decrypt an incoming request — malformed envelope, wrong key, corrupted/truncated payload — it responds:

```
400 Bad Request
```

Client handling: treat a 400 on an encrypted call as a signal to check the client-side encryption implementation (key correctness, IV format, envelope structure), not as a transient error. Do not blindly retry the same request; retrying an implementation bug will just produce the same 400.

Section 2: Authentication

2.1 Login and Token Fields

Login is performed via:

```
POST /api/Login/Login
```

On success, the response `Data` object contains two distinct token-like fields — **use the right one**:

Field	What it is	Use for auth?
`jwt_token`	An AES-256-GCM-encrypted JWT. Treat as an **opaque string** — do not attempt to decrypt or parse it client-side.	**Yes — this is the auth token.**
`tokens`	A separate, opaque session identifier used elsewhere in the system.	No — not used for the `Authorization` header.

```
json
```

```
{
  "Data": {
    "jwt_token": "eyJhbGciOi...opaque-encrypted-string...",
    "tokens": "some-other-opaque-session-id"
  }
}
```

Even though `jwt_token` is a JWT under the hood, it's encrypted (AES-256-GCM) before being sent to the client, so from the mobile client's perspective it is just an opaque token to store and echo back — no client-side decoding, decryption, or expiry-claim inspection.

2.2 Client Responsibilities

1. **Store `jwt_token` securely** after login — platform secure storage only (e.g., Keychain, EncryptedSharedPreferences, or equivalent secure enclave/keystore mechanism for the platform in use). Do not persist it in plain shared preferences, plist files, local storage, or logs.
2. **Send it on every subsequent authenticated API call** as a standard bearer token:

```
Authorization: Bearer <jwt_token>
```

3. **Treat `401 Unauthorized` as "session invalid or expired."** Standard flow:
   - Clear the locally stored token.
   - Redirect the user to the login screen.
   - Do not silently retry the request or loop on 401s.
4. **Do not implement a token refresh flow.** None exists yet. The token's effective lifetime currently matches the session (~4 hours), and once it expires, re-login is the only path back to an authenticated state. If this changes, the backend team will communicate a refresh mechanism separately.

2.3 401 Response Shape

When a call fails auth, the server returns a JSON body (itself unencrypted, since auth is checked independently of the encryption layer) shaped like:

```
json
{
  "Message": "...",
  "IsSuccess": false,
  "ResponseCode": 401
}
```

The client may surface `Message` directly to the user where appropriate, rather than showing a generic error string, since the server provides a human-readable reason.

Section 3: Combining Both Mechanisms

Encryption and authentication are independent layers and, once enforcement is turned on, will both be required on the same request where an endpoint calls for it:

- `Authorization: Bearer <jwt_token>` — present on every authenticated call.
- `X-Encrypted: true` + encrypted `payload` envelope — present only if that specific endpoint requires encryption.

Neither mechanism substitutes for the other: a valid bearer token does not exempt a request from encryption, and an encrypted payload does not exempt a request from needing a valid token.

3.1 End-to-End Example

Step 1 — Login (unencrypted, no token yet)

Request:

```
POST /api/Login/Login
Content-Type: application/json

{
  "username": "jdoe",
  "password": "••••••••"
}
```

Response:

```json
{
  "IsSuccess": true,
  "ResponseCode": 200,
  "Message": "Login successful",
  "Data": {
    "jwt_token": "eyJhbGciOi...opaque-encrypted-string...",
    "tokens": "some-other-opaque-session-id"
  }
}
```

The client stores `jwt_token` in secure storage (and ignores `tokens` for auth purposes).

Step 2 — Subsequent authenticated + encrypted call

The client encrypts its JSON body using the shared AES key and a freshly generated IV, builds the `payload` envelope, and sends:

Request:

```
POST /api/orders
Authorization: Bearer eyJhbGciOi...opaque-encrypted-string...
X-Encrypted: true
Content-Type: application/json

{
  "payload": "cmFuZG9tSVZieXRlcw==:c29tZUNpcGhlcnRleHRCeXRlcw=="
}
```

The server validates the bearer token, decrypts `payload` with the shared key, and processes the decrypted JSON. It then encrypts its own JSON response body the same way and returns:

Response:

```
json

{
  "payload": "YW5vdGhlcklWYnl0ZXM=:cmVzcG9uc2VDaXBoZXJ0ZXh0Qnl0ZXM="
}
```

The client decrypts `payload` (split on `:`, base64-decode both parts, AES-256-CBC decrypt with the shared key and the given IV, UTF-8-decode, parse JSON) to get the actual response data.

If the token had expired instead: the server would return `401` with the `Message` / `IsSuccess: false` / `ResponseCode: 401` shape from Section 2.3, and the client should clear the stored token and redirect to login as described in Section 2.2 — there is no refresh path.

Note: during the current rollout period (Section 0), the server will not actually reject calls missing either mechanism — the above describes the target contract to build toward and test against, not a hard gate today.

Open Items to Confirm with Backend Team

- Exact enforcement/cutover date for rejecting non-compliant calls.
- Whether a token refresh mechanism will be introduced in the future, and if so, when.
- Retry policy (if any) for transient (non-400/401) failures.

- Rate limiting behavior on auth or encrypted endpoints.
- Whether the login endpoint itself will ever require encryption.